Page No.
Date 24/6

Experiment No. 1. A.

Aim = Write a python program to take marks of 5 subject from the user. Calculate & print the total & average marks.

Program :

```
mark = [ ]
for i in range (1,6) :
    Marks = int (input ("Enter marks for subject : "))
    mark.append (Marks)

total = sum (mark)
avg = total / 5

print (f"\n Total marks = {total} ")
print (f" Average marks = {avg} ")
```

Output :

```
Enter the mark for sub : 22
Enter the mark for sub : 24
Enter the mark for sub : 34
Enter the mark for sub : 45
Enter the mark for sub : 56

Total marks = 181
Avg mark = 36.2
```

Conclusion :-
The program take marks of 5 subjects, calculates the total & average & display the result.

Experiment No. 1. B. 2416

Aim = Ask user to enter a number & print it's multiplication table up to 10 using a loop.

Program :

```
num = int (input ("Enter number to mul :"))
print (f"\n multiplication table {num}:")
for i in range (1, 11):
    print (f"{num} x {i} = {num * i}")
```

Output :

Enter number to mul : 2

Multiplication Table 2 :
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
2 x 4 = 8
2 x 5 = 10
2 x 6 = 12
2 x 7 = 14
2 x 8 = 16
2 x 9 = 18
2 x 10 = 20

Conclusion :
This Program take a number from user & prints it's multiplication table up to 10 using loop.

24/06/26

Experiment 1.C

Modify Ex-1 : part A to calculate max^n min^n marks & to calculate the grade of student

Grads are defin as greater than 75 is excellent.
Grade than equal 60 is good.
—"— —"— to 40 pass
Else fail.

Program =

```
subjects = ["Power sys", "control sys", "ele. machine"
            "python", "data science"]

mark = []
for subject in subjects:
    marks = float(input(f"Enter mark of {subjects}:
            "))
    mark.append(marks)

total = sum(mark)
avg = total / len(mark)
highest = max(mark)
lowest = min(mark)

print("\n---- REPORT ----")
for i in range(len(subjects)):
    print(subjects[i], ":", mark[i])

print("\n Total :", total)
```

```
print ("In AVG : ", avg)
print ("In High marks : ", highst
print ("In Lowest : ", lowest

if avg >= 75 :
    print ("Excellent")
elif avg >= 60 :
    print ("Good")
elif avg >= 40 :
    print ("Pass")
else :
    print ("Fail")
```

Output :

```
enter mark of power sys : 78
——— " ——— control sys : 89
——— " ——— ele machine : 82
——— " ——— python : 85
——— " ——— data science : 81

---- REPORT ----
Power sys : 78.0
Control sys : 89.0
ele machine : 82.0
python : 85.0
data science : 81.0

Total : 415.0
AVG : 83.6
High Marks : 89
Lowest : 78.0
Excellent.
```

Pass
27/09/25

Experiment No: 1 D

Aim =

Program =

```
def calculation_bill():
    if unit <= 100:
        bill = unit * 4
    elif uint <= 300:
        bill = (100 * 4) + ((unit - 100) * 6)
    else:
        bill = (100*4) + (200*6) + ((unit - 300) * 8)
    return bill
Unit = int(input("Enter units:"))
bill = calculation_bill()
print("Electric Bill = Rs", bill)
```

Output =

```
Enter Units : 5678
Electric bill : Rs 44624.
```

01/07/26

Experiment No: 2

Aim = To creat 1D & 2D arrays. using numpy & perform basic operation.

Program =

```
import numpy as np
arr1D = np.array([10, 20, 30, 40, 50])
arr2D = np.array([[1,2,3], [4,5,6]])
print("1D Arr:", arr1D)
print("2D Arr:", arr2D)

print("\n--- Basic Operation ---")
# slicing.
print("sliced 1D Arr:", arr1D[2:4])
# print("sliced 2D Arr:", arr2D[0:])
print("sliced 2D Arr (First Row):", arr2D[0:

# Mean.
print("Mean of 1D Arr:", np.mean(arr1D))
print("Mean of 2D Arr:", np.mean(arr2D))

# Max.
print("Max of 1D Arr:", np.max(arr1D))
print("Max of 2D Arr:", np.max(arr2D))

# std Deviation.
print("std Deviation of 1D Arr:", np.std(
    arr1D))

# Arthmatic
arr1 = np.array([1,2,3])
arr2 = np.array([4,5,6])
```

```
print("Arr 1:", arr1)
print("Arr 2:", arr2)

# ADD
print("ADD:", arr1 + arr2)
# SUB
print("SUB:", arr1 - arr2)
# MUL
print("MUL:", arr1 * arr2)

# Reshape
reshaped = np.array([1,2,3,4,5,6]).reshape(3,2)
print("Reshaped Arr:\n", reshaped)

# Indexing
print("Element at [1,0]", reshaped[1,0])
print("Second column:", reshaped[:,1])
```

Output =

```
1D Arr : [10 20 30 40 50]
2D Arr : [[1 2 3]
          [4 5 6]]

--- Basic Operation ---

Sliced 1D Arr (Index 1-3): [20 30 40]
Sliced 2D Arr (First Row): [[1 2 3]
                            [4 5 6]]
Mean of 1D Arr : 30.0
Mean of 2D Arr : 3.5
Max of 1D Arr : 50
Max of 2D Arr : 6
Std Deviation of 1D Arr : 14.142135

Arr 1 : [1 2 3]
Arr 2 : [4 5 6]
ADD : [5 7 9]
SUB : [-3 -3 -3]
MUL : [4 10 18]
Reshaped Arr :
 [[1 2]
  [3 4]
  [5 6]]

Element at [1,0] : 3
Second column : [[1 2]]
```

08/07/25

Experiment No. 3

Aim = Load csv file & fill empty cell with mean values

① Creating Data frame

Program :

```
import pandas as pd
name = ["John", "Henry", "Jack", "Mary"]
deg = ["MBA", "BBA", "BE", "BCom"]
scor = [90, 40, 80, 98]

firstdict = {'Name': name,
             'Degree': deg,
             'Score': scor}

firstdf = pd.DataFrame(firstdict)

firstdf
```

Output :

	Name	Degree	Score
0	John	MBA	90
1	Henry	BBA	40
2	Jack	BE	80
3	Mary	BCom	98

② Selecting columns using column Names :

```
firstdf [['Name', 'Score']]
```

o/p =

	Name	Score
0	Joha	90
1	Henry	40
2	Jack	80
3	Mary	98

(III) selecting colums using column index:

dfd[dfd.colums[2:4]]

dfd[dfd.colums[2:3]]

* firstdf[firstdf.colums[2:3]]

o/p =

	Degree	Score
0	MBA	90
1	BBA	40
2	BE	80
3	BCom	98

(IV) Creating Data frame from Dictionary

→ Give index.

firstdf = pd.DataFrame(firstdict, index = ["a", "b", "c", "d"])

o/p =

	Name	Degree	Score
a	Joha	MBA	90
b	Henry	BBA	40
c	Jack	BE	80
d	Mary	BCom	98

(V) Selecting 2 rows & 4 column 'Name' to 'Score'

o/p =

	Name	Degree	Score
a	Joha	MBA	90
b	Henry	BBA	40

function =

firstdf.loc['a':'b', 'Name':'Score']

(VI) Using iloc for slicing rows & colums.

firstdf.iloc[0:2, 2:3]

o/p =

	Degree	Score
a	MBA	90
b	BBA	40

(VIII) Adding columns using assign()

Add new dataframes as dataframes
(firstdf2)

Code:

```
import pandas as pd
name = ["Joha", "Henry", "Jack", "Mary"]
deg = ["MBA", "BBA", "BE", "BCom"]
scor
```

```
firstdf2 = pd.DataFrame(first dict)
firstdf2.assign(Height = [5.1, 6.2, 5.1, 5.2])
```

o/p:

	Name	Degree	Score	Height
0	Joha	MBA	90	5.1
1	Henry	BBA	40	6.2
2	Jack	BE	80	5.1
3	Mary	BCom	98	5.2

(VII) Adding colums using insert().

```
firstdf2.insert(2, "Height", [5.1, 6.2, 5.1, 5.2],
                allow_duplicates = True)
```

o/p:

	Name	Degree	Height	Score
0	Joha	MBA	5.1	90
1	Henry	BBA	6.2	40
2	Jack	BE	5.1	80
3	Mary	BCom	5.2	98

(IX) Adding Rows Using Append

```
df2 = {'Name': 'Tom', 'Degree': 'BE', 'Score': 99}
firstdf2.append(df2, ignore_index=True)
```

o/p:

	Name	Degree	Score
0			
1			
2			
3			
4	Tom	BE	99

(X) Concatenate by rows

```
data = {'Name': ['Jack', 'Mary'],
        'Degree': ['BE', 'MCA'],
        'Score': [88, 65]}
df3 = pd.DataFrame(data)
```

```
df4 = pd.concat([df2, df3], axis=0)
df4
```

o/p:

	Name	Degree	Score	Age
0	John	MBA	90	NaN
1	Henry	BBA	40	-11-
2	Jack	BE	80	-11-
3	Mary	BCom	98	21.0
0	Jack	BE	88	43.0
1	Mary	MCA	65	

15/07/26

Aim: Load csv file to dataframe & Handle missing values

Program:

```
import pandas as pd
import Numpy as np

df = pd.DataFrame({'ID': [1,2,3,4,5,6],
                   'Score': [50,55,np.NaN,1000,65,70],
                   'Age': [20,21,np.nan,22,23,24]
                   })
print("Original DataFrame:\n", df)
#2.
df['Score'].fillna(df['Score'].mean(),
                   inplace=True)

df['Age'].fillna(df['Age'].mean(),
                 inplace=True)

print("\nAfter fill missing values:\n", df)
```

o/p = Original DataFrame:

	ID	Score	Age
0	1	50.0	20.0
1	2	55.0	21.0
2	3	NaN	NaN
3	4	1000.0	22.0
4	5	65.0	23.0
5	6	70.0	24.0

After fill missing value:

	ID	score	Age
0	1	50.0	40.0
1	2	55.0	21.0
2	3	248.0	22.0
3	4	1000.0	22.0
4	5	65.0	25.0
5	6	70.0	24.0

(ii) Program:

```
import pandas as pd
import numpy as np
df = pd.read_csv("data.csv")
print(df)

df['Marks'].fillna(df['Marks'].mean(), inplace
                   =True)
df['Attendance']:fillna(df['Attendance'].mean(),
                        inplace=True)
print("\n After fill missing value:\n", df)
```

Output =

	Roll No	Name	Department	Mark	Atten
0	101	Amit	Ele	85.0	90
1	102	Sneha	Ele	NaN	88.0
2	103	Shridhar	Ele	92.0	94.0
3	104	Rushi	CS	85.0	34.0
4	105	Vishal	CS	78.0	Nan

After fill missing value:

Roll no	Name	Department	Mark	Attendance
101	Amit	Ele	85.0	90.0
102	Sneha	Ele	70.0	88.0
103	Shridhar	Ele	92.0	94.0
104	Rushi	CS	85.0	34.0
105	Vishal	CS	78.0	76.5

(iii) Program after Dropping missing value:

```
import pandas as pd
import numpy as np
df = pd.read_csv("data.csv")
print(df)
df_drop = df.dropna()
print("\n Dropping Missing value:\n", df_drop
```

Output = (After Dropping Missing value:)

Roll No	Name	Department	Marks	Attendar
101	Amit	Ele	85.0	90.0
103	Shridhar	Ele	92.0	94.0
104	Rushi	CS	85.0	34.0

Pwt
22/07/26

27/7/26

Experiment No: 3

Program:

```
import pandas as pd
import numpy as np

df = pd.DataFrame({'ID': [1,2,3,4,5],
                   'score': [50,55,0,79,84],
                   'Age': [20,22,23,24,25]})

print(df)

Q1 = df['score'].quantile(0.25)
Q3 = df['score'].quantile(0.75)
IQR = Q3 - Q1

# Boundaries
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
print(lower_bound)
print(upper_bound)

# Filter
df_cleaned = df[(df['score'] >= lower_bound)
                & (df['score'] <= upper_bound)]

print(df_cleaned)
```

output:

```
11.5
127.5
```

	ID	score	Age
0	1	50	20
1	2	55	22
3	4	79	34
4	5	84	25

```
# df2 = pd.DataFrame({'ID': [1,2,3,4],
                      'Department': ['HR','IT','Sale','CS']})

merged_df = pd.merge(df, df2, on='ID',
                     how='inner')

print(merged_df)
```

output =

	ID	score	Age	Department
0	1	50	20	HR
1	2	55	22	IT
2	3	56	23	Sale
3	4	79	34	CS

Page No.
Date 29/7/26

Experiment No. 4.

Aim:

① Groupby

program:

```
import pandas as pd
import numpy as np

data = pd.DataFrame({'Region': ['North', 'South',
        'North', 'East', 'South', 'North'],
    'Sales': [100, 150, 120, 200, 180, 130]})

print("Original sale DF:\n", data)

# Groupby
region_sale = data.groupby('Region')['Sales']
    .agg(['sum', 'mean'])
print("\nTotal + Avg sales by region:\n",
        region_sale)
```

Output:

Original sales DF:

0	North	100
1	South	150
2	North	120
3	East	200
4	South	180
5	North	130

Total + Avg Sales by region:

Region	Sum	mean
East	200	200.0
North	350	116.6
South	330	165.0

29/07/26

Page No.
Date 5/8/26

Experiment No: 4 B.

Aim: Normalize sales column using transform() & display the originals transformed data side by side.

program:

```
import pandas as pd
import numpy as np

data = {'Region': ['North', 'South', 'North', 'East',
                   'South', 'North'],
        'Sales': [100, 150, 120, 200, 180, 130]}

sales_df = pd.DataFrame(data)
print("Original Sales DF:")
display(sales_df)

sales_df2 = sales_df.copy()

sales_df2['Sales_Normalized'] = sales_df['Sales'].transform(lambda x: (x-x.min())/
                                (x.max()-x.min()))
display(sales_df[['Sales']].join(sales_df2[['Sales_Normalized']]))
```

Output:

	Sales	Sales_Normalized
0	100	0.0
1	150	0.5
2	120	0.2
3	200	1.0
4	180	0.8
5	130	0.3

Result
05/08/26

Aim :

program :

```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.DataFrame({"student": ["A", "B", "C", "D",
                               "E", "F", "G", "H"],
                   "Math": [78, 85, 90, 67, 88, 72, 95, 80],
                   "Science": [82, 79, 94, 70, 86, 75, 98, 85],
                   "English": [74, 81, 89, 65, 90, 70, 92, 78]})
print(df)

print("\n Summary statistics :")
print(df.describe())
```

student	Math	Science	English
A	78	82	74
B	85	79	81
C	90	94	89
D	67	70	65
E	88	86	90
F	72	75	70
G	95	98	92
H	80	85	78

Summary statistics :

	Math	Science	English
Count	8.0	8.0	8.0
mean	81.875	83.62	79.87
std	9.43	9.30	7.93
min	67.0	70.0	65.0
25 %	76.5	78.6	73.0
50 %	82.5	83.5	79.0
75 %	88.5	88.0	89.25
100 %	95.0	98.0	92.0

12/08/26

Experiment No. 5

Aim: To perform EDA on real time data to calculate correlation matrix.

Code:

```
import pandas as pd.
speed = [20, 30, 40, 50, 60].
travel_time = [60, 50, 40, 30, 20].
speed_series = pd.series(speed)
time_series = pd.series(travel_time).
correlation = speed_series.corr(time_series).
print("Correlation coefficient =", correlation)
```

Correlation Coefficient = -1.0.

```
import pandas as pd.
study_hourse = [1, 2, 3, 4, 5]
shoe_size = [8, 6, 7, 7, 8]
x = pd.series(study_hours)
y = pd.series(shoe_size).
correlation = x.corr(y)
print("Correlation coefficient =", correlation)
```

Correlation coefficient = 0.138675

PWC
13/08/20

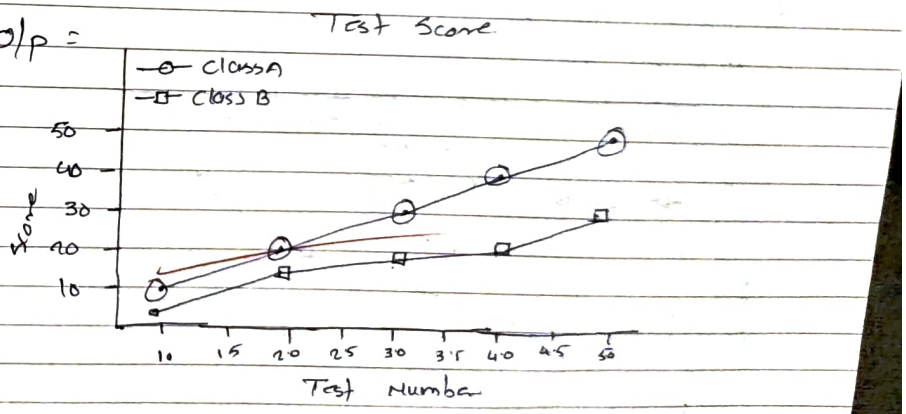
Experiment No. 6

Aim = Create line, bar, pie & scatter plots using matplotlib with proper labels & legends.

Code =

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4, 5]
y1 = [10, 20, 30, 40, 50]
y2 = [5, 15, 20, 25, 35]
plt.figure(figsize=(6, 4))
plt.plot(x, y1, label='Class A', marker='o')
plt.plot(x, y2, label='Class B', marker='s')
plt.title("Test Score")
plt.xlabel("Test Number")
plt.ylabel('Score')
plt.legend()
plt.grid(True)
plt.show()
```

o/p =

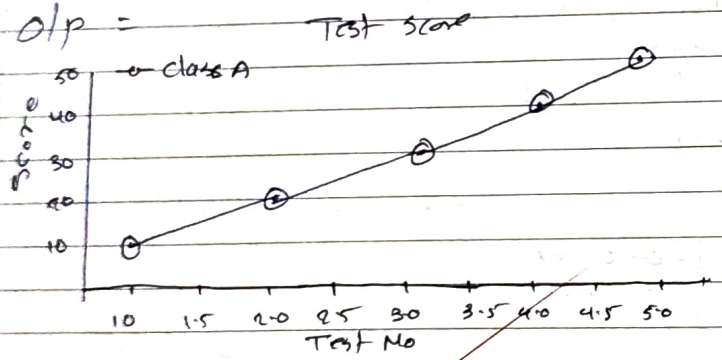


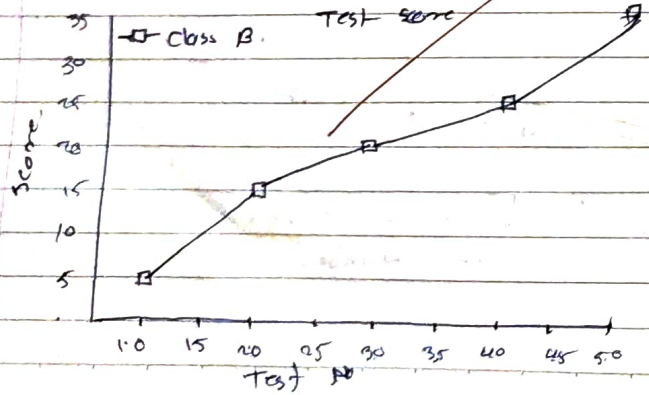
Code =

```
plt.figure(figsize=(6,4))
plt.plot(x, y1, label='Class A', marker='o')
plt.title('Test score')
plt.xlabel('Test No')
plt.ylabel('score')
plt.legend()
plt.grid(True)
plt.show()

plt.figure(figsize=(6,4))
plt.plot(x, y2, label='Class B', marker='s')
plt.show()
```

o/p =

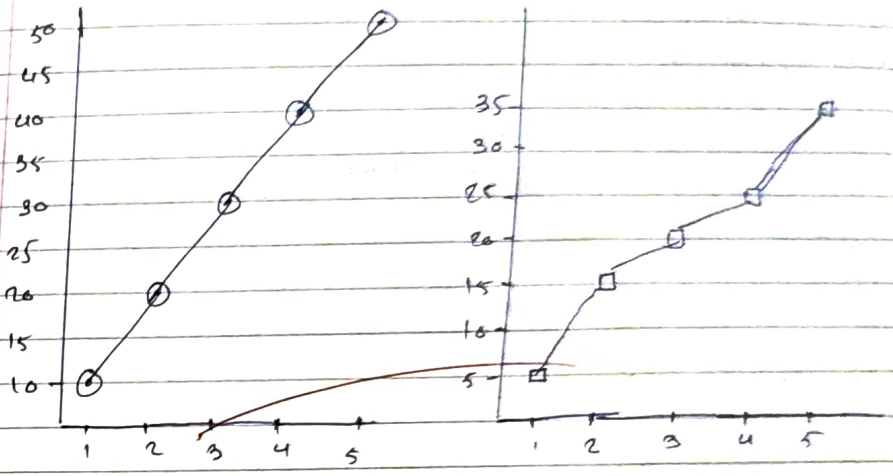




Code = (sub-plot)

```
fig, (ax1, ax2) = plt.subplots(1,2, figsize=(6,4))
```

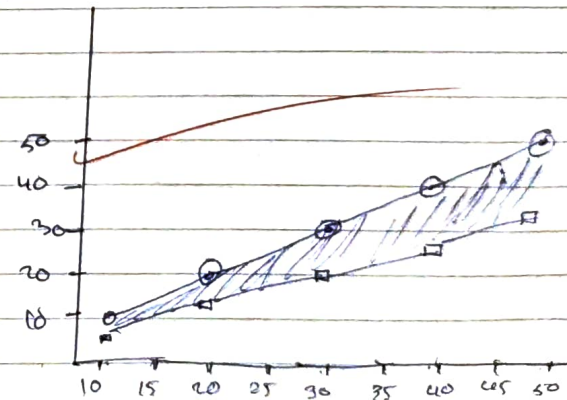
o/p =



Code = (Fill_bet^n)

```
plt.fill_between(x, y1, y2, colour='Black', alpha=0.3, label='Diff')
```

o/p =

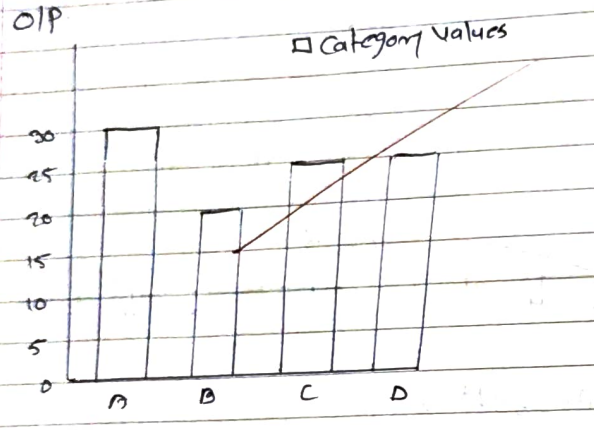


Code :(Bar)

```
import matplotlib.pyplot as plt
x = ['A', 'B', 'C', 'D']
y = [30, 20, 25, 25]
plt.bar (x, y, color = 'Black', label = 'Category values')

plt.legend()
plt.show().
```

O/P

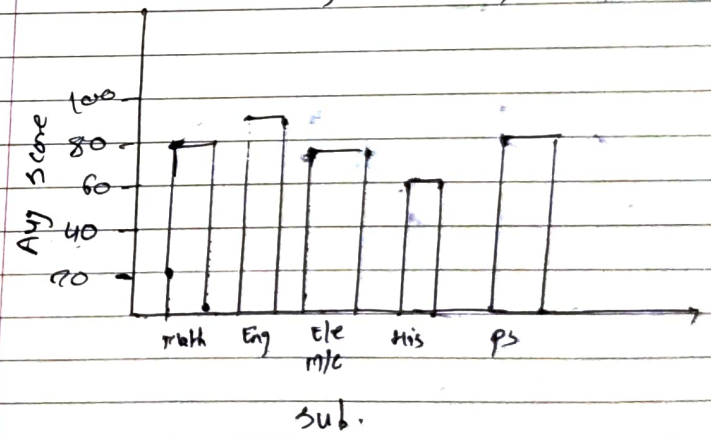


Aim =

Program =

```
import matplotlib.pyplot as plt
x = ['Math', 'Eng', 'Ele. M/c', 'History', 'PS']
y = [80, 90, 75, 60, 85]
plt.figure (figsize = (6,4))
plt.bar (x, y, color = 'skyblue', edgecolor =
        'black', width = 0.5)
plt.title ('Avg Score by Sub')
plt.xlabel ('Sub')
plt.ylabel ('Avg Score')
plt.show ()
```

Output =

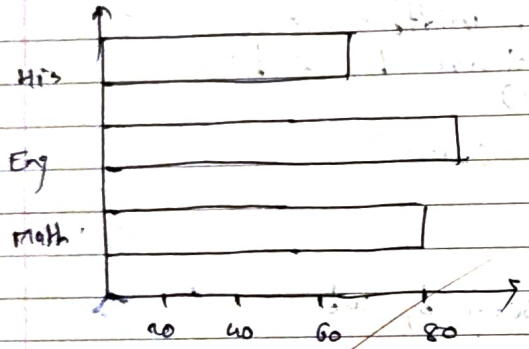


(II) Horizontal bar =

```
plt.barh (x, y, color = 'skyblue', edgecolor
          = 'black', height = 0.5)

plt.title ('—II—')
plt.xlabel ('Avg Score')
plt.ylabel ('Sub')
```

Output =



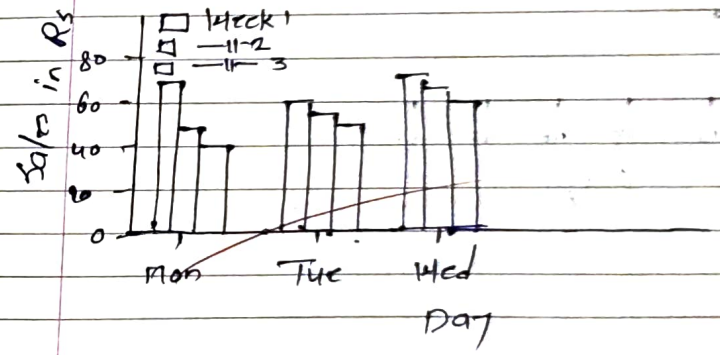
(IIA) Group Bar =

```
import pandas as pd
import matplotlib.pyplot as plt

data = { 'Day' : ['Mon', 'Tue', 'Wed', 'Thu',
                                    'Fri'],
    'Week 1' : [50, 60, 70, 40, 70],
    'Week 2' : [45, 55, 65, 75, 85],
    'Week 3' : [40, 50, 60, 70, 80] }

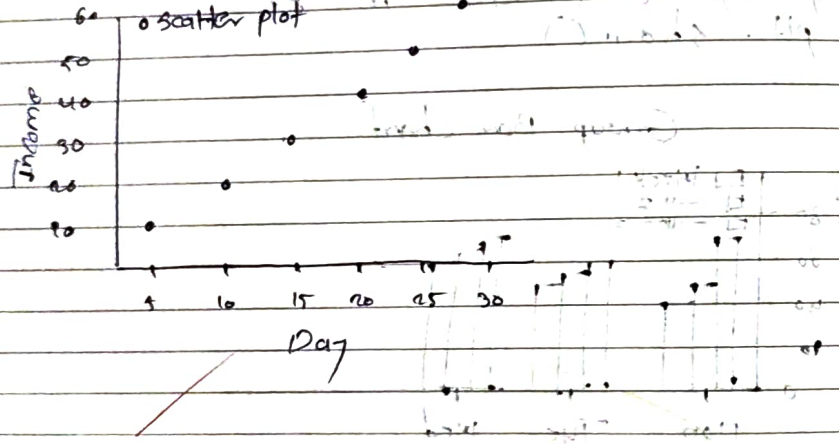
df = pd.DataFrame (data)
df.plot (kind = 'bar', x = 'Day', title = 'Grou
                        ped Bar chart')
plt.ylabel ('Sales in Rs')
plt.show ()
```

Group Bar chart



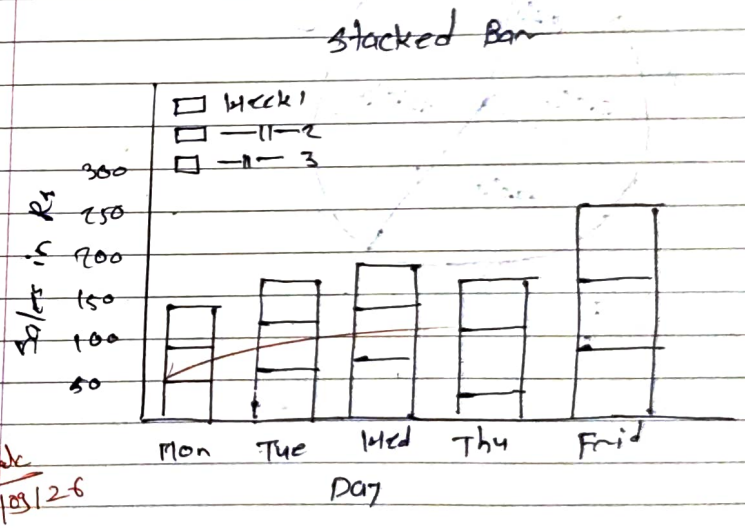
(IV) Scatter plot :

```
import matplotlib.pyplot as plt
x = [5, 10, 15, 20, 25, 30]
y = [10, 20, 30, 40, 50, 60]
plt.scatter(x, y, color='red', marker='o',
            label='Scatter Plot')
plt.xlabel('Day')
plt.ylabel('Income')
plt.legend()
plt.show()
```



(VI) Stacked Bar chart :

```
import pandas as pd
import matplotlib.pyplot as plt
data = {
    'Day': ['Mon', 'Tue', 'Wed', 'Thu', 'Fri'],
    'Week1': [50, 60, 70, 40, 90],
    'Week 2': [45, 55, 65, 75, 85],
    'Week 3': [40, 50, 60, 70, 80]
}
df = pd.DataFrame(data)
df.plot(kind='bar', x='Day', stacked=True,
        title='Stacked Bar')
plt.ylabel('Sales in Rs')
plt.show()
```

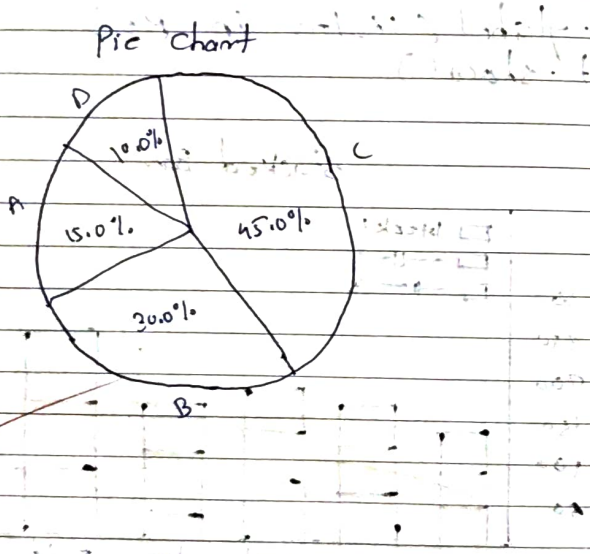


09/09/26

Pie chart =

```
import matplotlib.pyplot as plt.
colors = ['gold', 'yellowgreen', 'lightcoral', 'lightskyblue']
labels = ['A', 'B', 'C', 'D']
size = [15, 30, 45, 10]
plt.pie(size, labels = labels, colors = colors,
        autopct = '%1.1f%%', shadow = True,
        startangle = 140)
plt.title('Pie chart')
plt.axis('Equal')
plt.show()
```

Output =



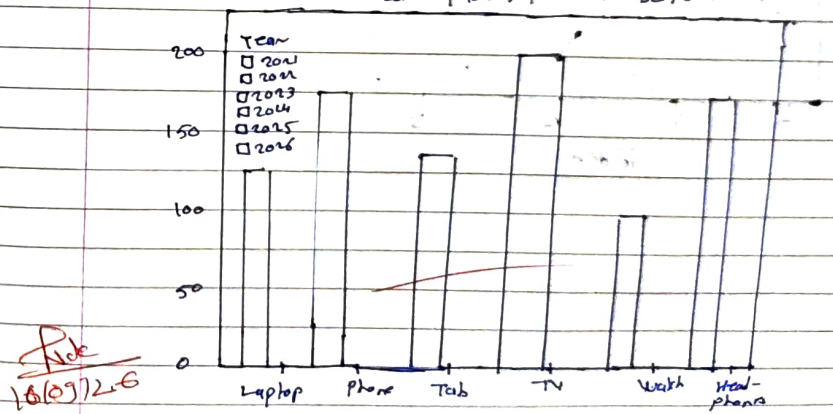
16/09/26

bar plot Using seaborn library.

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

data = {
'Product' : ['Laptop', 'Phone', 'Tab', 'TV', 'Watch', 'Headphones'],
'Year' : [2021, 2022, 2023, 2024, 2025, 2026],
'Sales' : [120, 180, 150, 220, 190, 250]
    }

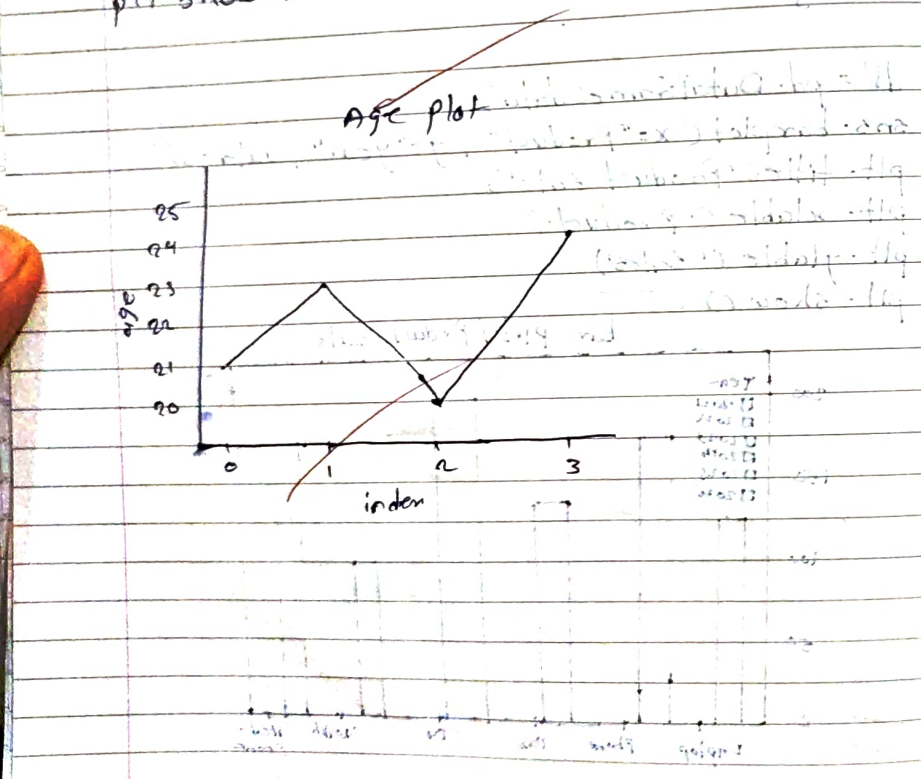
df = pd.DataFrame(data)
sns.barplot(x="Product", y="year", data=df)
plt.title('Product sale')
plt.xlable('Product')
plt.ylable('Sales')
plt.show()
```



16/09/26

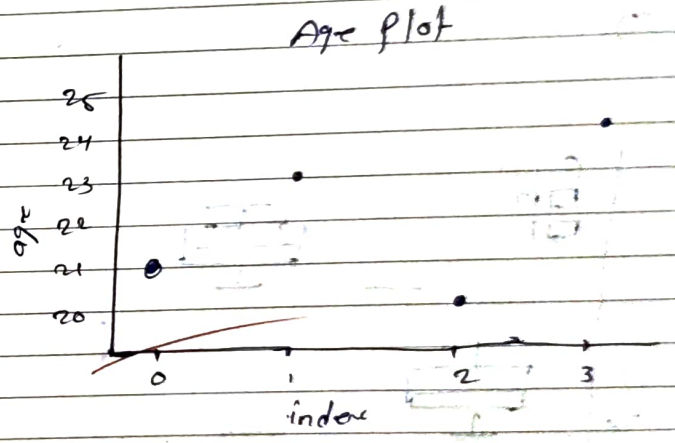
Line plot =

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
data = {'Name':['Anosh','Nawaaf','Saket','Prashik'],
        'Age':[21,23,20,24]}
df = pd.DataFrame(data)
sns.lineplot(x=df.index, y='Age', data=df)
plt.title('Age plot')
plt.xlabel('index')
plt.ylabel('Age')
plt.show()
```



Scatter plot

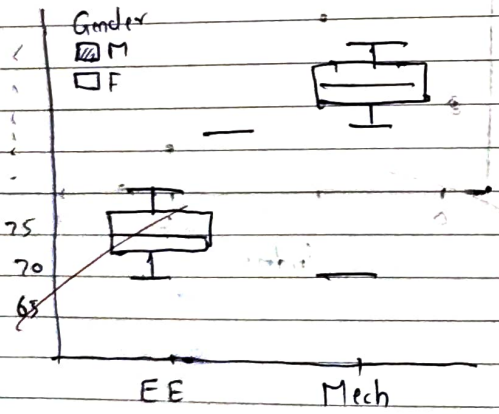
```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
data = {'Name':['Anosh','Nawaaf','Saket','Prash
                ik'],
        'Age':[21,23,20,24]}
df = pd.DataFrame(data)
sns.scatterplot(x=df.index, y='Age', data=df)
plt.title('Age plot')
plt.xlabel('index')
plt.ylabel('Age')
plt.show()
```



Box plot =

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
data = {'Department': ['EE', 'EE', 'ME', 'ME', 'EE',
                       'ME'],
 'Gender': ['M', 'F', 'M', 'F', 'M', 'F'],
 'Marks': [78, 70, 88, 65, 92, 79]}
df = pd.DataFrame(data)

sns.boxplot(data=df, x='Department', y='Marks',
            hue='Gender')
plt.title('Distribution of Marks')
plt.xlabel('Department')
plt.ylabel('Marks')
plt.show()
```

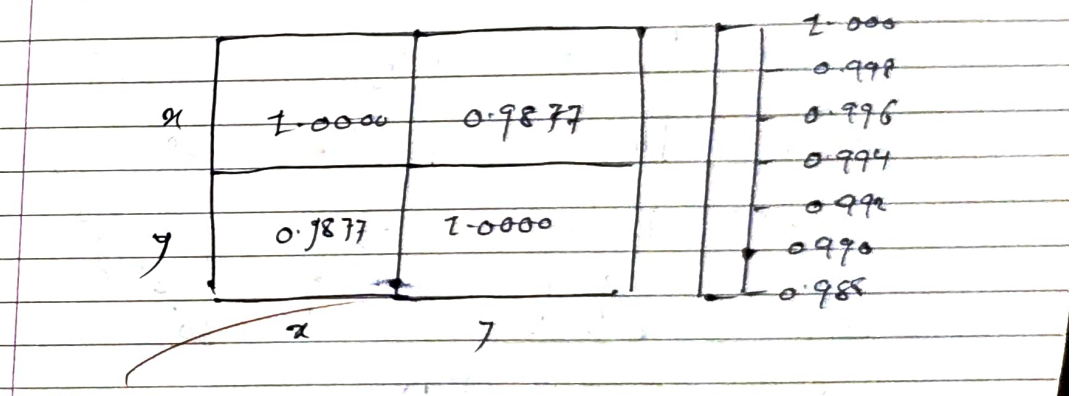


Heatmap

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

x = [10, 12, 14, 16, 18, 20]
y = [5, 8, 11, 14, 16, 17]

df = pd.DataFrame({"x": x, "y": y})
corr = df.corr()
sns.heatmap(corr, annot=True, fmt=".4f",
            cmap="Blues")
plt.title("Correlation Heatmap")
plt.show()
```



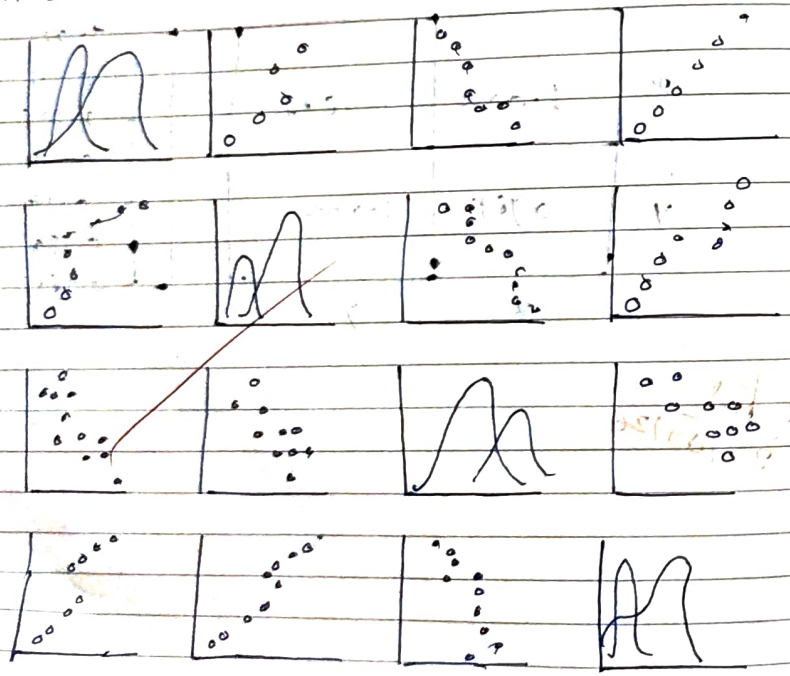
Ans
23/09/26

Pair plot

```
import matplotlib.pyplot as plt
       pandas as pd
       seaborn as sns

data = {'x' :[2.5,5.0,1.0,3.5,6.0,2.0,4.5,5.5,1.5,4.0],
        'y' :[75,90,60,85,95,70,88,92,65,82],
        'z' : [7,6,8,7,5,8,6,6,9,7],
        'A' : [65,88,50,78,75,60,85,90,55,80],
        'B' : ['P','P','F','P','P','P','P','P','F','P'] }

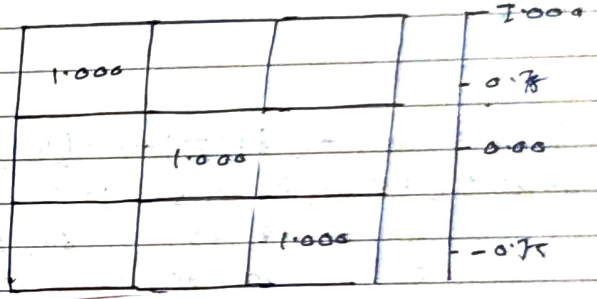
df = pd.DataFrame(data)
sns.pairplot(df, hue='B', palette="set1", diag_kind="kde")
plt.suptitle("Pair plot", y=1.02)
plt.show()
```



Heatmap

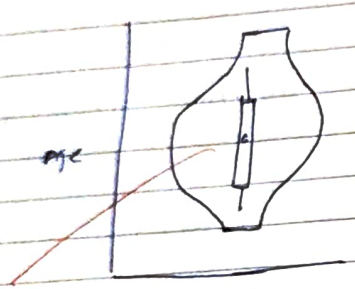
```
import matplotlib.pyplot as plt
       pandas as pd
       seaborn as sns

data = {'x' : [2.5,5.0,1.0,3.5,6.0,2.0,4.5,5.5,1.5,4.0],
        'y' : [75,90,60,85,95,70,88,92,65,82],
        'z' : [7,6,8,7,5,8,6,6,9,7]}
df = pd.DataFrame(data)
corr = df.corr()
sns.heatmap(corr, annot=True, fmt=".3f", cmap="Red")
plt.show()
```

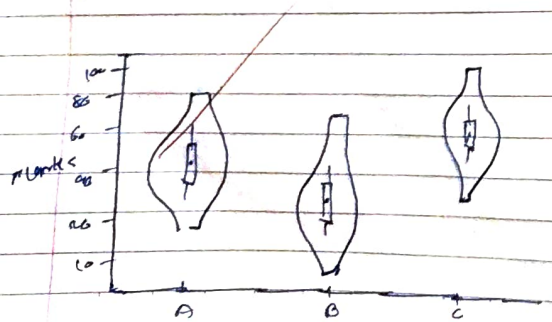


violineplot

```
data = {'Name':['Arsh','Ram','yash','Raj'],
        'Age':[21, 22, 20, 25, 23]}
df = pd.DataFrame(data)
sns.violinplot(y='Age', data=df)
plt.show()
```




```
date = {'class':['A']*8 + ['B']*8 + ['C']*8,
        'Mark':[55,60,62,65,68,70,72,75,
                40,45,48,43,47,60,70,
                50,85,75,35,70,75,100]}
df = pd.DataFrame(data)
sns.violinplot(x='class', y='Mark', data=df, inner='box')
plt.title("violine plot")
plt.show()
```



30/09/26

7/10/26

Code:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

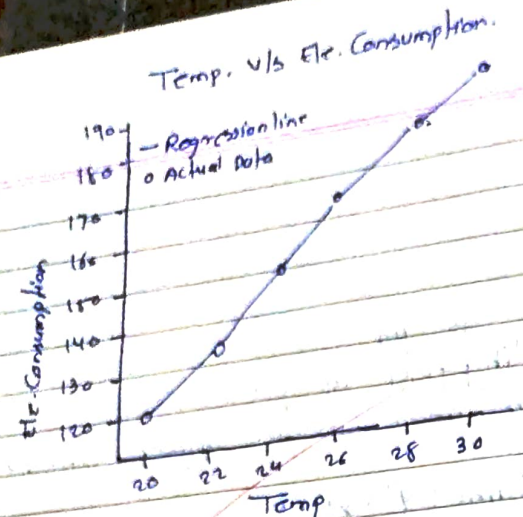
df = pd.DataFrame({'x':[20, 22, 24, 26, 28, 30],
                   'y':[120, 135, 150, 165, 180, 195]})

a = df[['x']]
b = df[['y']]
model = LinearRegression()
model.fit(a, b)

prediction = model.predict([[32]])
print("predicted at 32°", prediction[0], "Kwh")

plt.scatter(a, b, label='Actual data')
plt.plot(a, model.predict(a), label="Regression Line")

plt.xlabel('Temp')
plt.ylabel('Ele. Consumption')
plt.title('Temp v/s Ele. Consumption')
plt.legend()
plt.show()
```




```
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

df = pd.DataFrame ({'study-Hr': [1,2,3,4,5,6,7,8,9],
                    'Result': ['F', 'F','F','F','F','P','P','P','P','P']
                    })

x = df[['study-Hr']]
y = df['Result']

x_train, x_test, y_train, y_test = train_test_split(
        x, y, test_size=0.3, random_state=42 )

model = LogisticRegression()
model.fit(x_train, y_train)
y_pred = model.predict(x_test)

print("Actual Result:", list(y_test))
print("Predicted Result:", list(y_pred))
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
new_student = [[6]]
prediction = model.predict(new_student)

print("Prediction for 6 study Hr:", prediction[0])
```

Actual Result : ['P', 'F', 'P']
Predicted Result : ['P', 'F', 'F']
Accuracy : 0.6666
Prediction for 6 study Hrs : Pass

Pook
07/10/25

14/10/26

program:

```
from sklearn.neighbors import KNeighborsClassifier

X = [[1], [2], [3], [5], [6], [7]]
Y = ['F', 'F', 'F', 'P', 'P', 'P']
model = KNeighborsClassifier(n_neighbors = 3)
model.fit(X, Y)
prediction = model.predict([[4]])
print("Predicted Result :", prediction[0])
```

Prediction Result : F

```
from sklearn.neighbors import KNeighborsClassifier

X = [[2], [3], [4], [7], [8], [9]]
Y = ['N', 'N', 'N', 'F', 'F', 'F']
Model = KNeighborsClassifier(n_neighbors = 3)
model.fit(X, Y)
result = model.predict([[7.5]])
print("Motor condition", result[0])
```

Motor condition : F

Pook
14/10/26